CORRIGÉ · PROBLÈME · ANNALES SPÉ

Chaînes, antichaînes et tas

Énoncé

Langage OCaml exclusivement. On représente une relation sur $V = \{v_0, \dots, v_{n-1}\}$ par une matrice `r : bool array array` de taille $n \times n$, avec `r.(i).(j)` vrai si et seulement si $v_i \trianglelefteq v_j$ (`type order = bool array array`).

Une chaîne est une partie C telle que deux éléments quelconques c, c' vérifient $c \trianglelefteq c'$ ou $c' \trianglelefteq c$ (représentée par une `int list`). Code donné :

```
let rec is_chain (r:order) (c:int list) : bool =
  match c with
  | [] -> true
  | v::cc -> List.for_all (fun x -> x > v || x < v) cc
             && is_chain r cc
```

1. Confirmer ou réfuter : `is_chain r c` teste si `c` est une chaîne pour `r`. Dans le second cas, corriger la fonction.
2. Complexité de `is_chain` en fonction de la longueur γ de `c`.

On veut `is_chain_bis` en $O(\gamma \log \gamma)$ avec une structure mutable de capacité bornée : `init : order -> int -> hp`, `is_empty : hp -> bool`, `push : hp -> int -> bool`, `pop : hp -> bool` ; `push` et `pop` peuvent échouer, ce que signale leur valeur de retour. La fonction insère tous les éléments de `c` puis les retire tous, et renvoie la conjonction des succès.

1. Compléter la description du type `hp` en nommant une structure de données (avec son invariant) pour que `is_chain_bis` respecte la spécification en $O(\gamma \log \gamma)$. Justifier.

Une antichaîne est une partie dont les éléments sont deux à deux incomparables.

1. Modifier `is_chain` en `is_antichain` ; en déduire sa complexité.
2. Confirmer ou réfuter l'existence d'une fonction `is_antichain_bis` en $O(\alpha \log^2 \alpha)$, où α est la longueur de la liste.

Décoder l'énoncé

Ce que l'énoncé te donne.

- `x > v || x < v` compare les **numéros** des sommets, pas les éléments pour la relation `r`.
- La transitivité permet de déduire des comparaisons sans les tester toutes.

Ce qu'on te demande. Tester chaînes et antichaînes, et voir pourquoi on peut aller vite pour les unes et pas pour les autres.

Corrigé

Question 12

Étape 1. Q12. La fonction is_chain est fausse

Réfutation : la fonction compare les entiers `x` et `v` avec l'ordre usuel $\leqslant$, et non avec la relation `r` (qui n'est même pas utilisée). Pour deux sommets distincts, `x > v || x < v` est toujours vrai : `is_chain r c` renvoie `true` pour toute liste sans doublon.

Contre-exemple : l'ordre « égalité » sur $V = \{v_0, v_1\}$ (`r` = matrice identité) ; $\{v_0, v_1\}$ n'est pas une chaîne, mais `is_chain r [0;1]` renvoie `true`.

Correction : tester la comparabilité pour `r`.

```
let rec is_chain (r:order) (c:int list) : bool =
  match c with
  | [] -> true
  | v :: cc -> List.for_all (fun x -> r.(v).(x) || r.(x).(v)) cc
               && is_chain r cc
```

Preuve (récurrence sur la longueur) : la liste vide est une chaîne. Une liste `v :: cc` est une chaîne si et seulement si `v` est comparable à chaque élément de `cc` (test du `for_all`) et `cc` est une chaîne (appel récursif, correct par hypothèse de récurrence) : ce sont exactement toutes les paires.

Question 13

Étape 1. Q13. Complexité de is_chain

Sur une liste de longueur ℓ, `for_all` fait au plus $\ell - 1$ tests en $O(1)$, puis l'appel récursif porte sur une liste de longueur $\ell - 1$. Dans le pire cas (une chaîne : aucun arrêt anticipé) :

$$\sum_{\ell=1}^{\gamma} (\ell - 1) = \frac{\gamma(\gamma - 1)}{2}.$$

Complexité : $O(\gamma^2)$.

Question 14

Étape 1. Q14. Un tas binaire qui échoue sur les incomparables

Structure : `hp` est un **tas binaire** (file de priorité) de capacité γ, rangé dans un tableau, dont les comparaisons se font avec `r`. Invariant de bonne constitution : pour tout nœud p de fils f, on a $v_p \trianglelefteq v_f$ (le parent est plus petit que ses fils pour `r`).

Opérations :

- `push q i` : on place i en dernière position et on le fait remonter. Chaque comparaison avec le parent utilise `r` ; si les deux éléments sont **incomparables**, `push` renvoie `false`.
- `pop q` : on retire la racine, on met le dernier élément à la racine et on le fait descendre en le comparant à ses fils (et les fils entre eux). Si une comparaison porte sur deux incomparables, `pop` renvoie `false`.

Correction :

- si `c` est une chaîne, toutes les comparaisons réussissent : la fonction renvoie `true` ;
- si la fonction renvoie `true`, l'invariant est vrai à chaque étape. La racine est alors plus petite que tous les éléments du tas (transitivité le long des branches), et le nouvel élément remonté à la racine après un `pop` était un descendant de l'ancienne racine. Les éléments sortent donc dans l'ordre $x_1 \trianglelefteq x_2 \trianglelefteq \cdots \trianglelefteq x_\gamma$; par transitivité, deux éléments quelconques sont comparables : `c` est une chaîne ;
- la fonction ne renvoie `false` que si elle a trouvé deux éléments de `c` incomparables, auquel cas `c` n'est pas une chaîne.

Complexité : γ insertions et γ retraits, chacun en $O(\log \gamma)$ comparaisons (hauteur du tas) : $O(\gamma \log \gamma)$. C'est un tri par tas qui vérifie au passage que tout est comparable.

Question 15

Étape 1. Q15. Tester une antichaîne

```
let rec is_antichain (r:order) (a:int list) : bool =
  match a with
  | [] -> true
  | v :: aa -> List.for_all (fun x -> not r.(v).(x) && not r.(x).(v)) aa
               && is_antichain r aa
```

La structure du code est identique à celle de `is_chain` (Q12) : seul le prédicat testé change, toujours en $O(1)$. Le décompte de Q13 s'applique tel quel : complexité $O(\alpha^2)$.

Question 16

Étape 1. Q16. Pas d'algorithme en $\alpha \log^2 \alpha$

Réfutation : tout algorithme correct doit consulter `r` pour **chacune** des $\frac{\alpha(\alpha-1)}{2}$ paires, donc faire $\Omega(\alpha^2)$ opérations, ce qui est plus que $O(\alpha \log^2 \alpha)$.

Argument d'adversaire : on part de l'ordre « égalité » ($v_i \trianglelefteq v_j \iff i = j$), pour lequel la liste `a` est une antichaîne, et l'algorithme doit répondre `true`. Supposons qu'il ne consulte jamais `r.(a).(b)` ni `r.(b).(a)` pour une certaine paire $\{a, b\}$ de la liste. On change alors `r` en ajoutant seulement $v_a \trianglelefteq v_b$. C'est encore une relation d'ordre (réflexive, antisymétrique, et transitive car il n'y a pas d'autre relation à enchaîner). L'algorithme fait exactement les mêmes lectures, donc répond encore `true`, alors que `a` n'est plus une antichaîne : contradiction.

Pourquoi c'est différent pour les chaînes : dans une chaîne, la transitivité permet de déduire des comparaisons sans les lire (c'est ce qu'exploite le tas). Pour l'incomparabilité, rien ne se déduit.

Le piège

En Q12, croire la fonction correcte parce qu'elle « compare » : elle compare les numéros des sommets avec $\leqslant$, et non les éléments pour l'ordre `r`.

La méthode

Borne inférieure de complexité : argument d'adversaire. Si l'algorithme ne lit pas une donnée, on la modifie sans qu'il s'en aperçoive.

Rappels de cours utilisés

- **Relation d'ordre.** Réflexive ($x \trianglelefteq x$), antisymétrique ($x \trianglelefteq y$ et $y \trianglelefteq x$ donnent $x = y$), transitive ($x \trianglelefteq y \trianglelefteq z$ donne $x \trianglelefteq z$). Une **chaîne** est une partie où deux éléments sont toujours comparables ; une **antichaîne**, une partie où deux éléments distincts ne le sont jamais.
- **Tas binaire.** Arbre binaire presque complet rangé dans un tableau (fils de i en $2i+1$ et $2i+2$), avec l'invariant : **tout nœud est plus petit que ses fils**. Insertion (remontée) et retrait de la racine (descente) en $O(\log n)$ comparaisons.